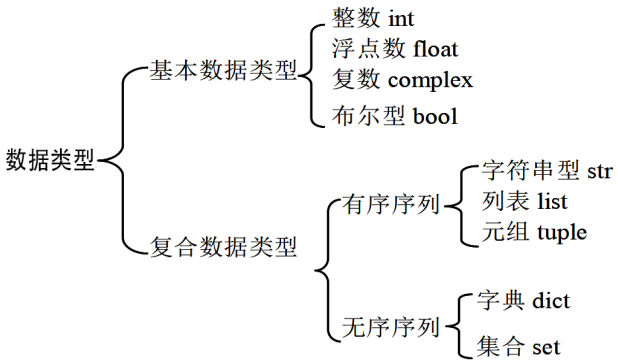
Python提供的内置数据类型



变量

◆ **变量命名**：必须是一个**合法的标识符**

变量命名遵循以下约定俗成的**规范**：

- 见名知义，以提高程序的可读性
- 单词之间用下划线连接，或者每个单词首字母大写
- 值不变的尽量用大写字母命名，例如：PI=3.14
- 标识符是**严格区分字母的大小写**
- 函数名可以作为变量名，但尽量不这样使用

如: max=65535;　int=2;　float=1.5

（3）链式赋值　　**作用**：给多个变量赋值

```
x=y=z=1        #x、y、z三个变量获得相同值1
```

（4）序列解包赋值　　**作用**：给不同的变量分别赋值

```
x,y,z=1,2,3    #x、y、z 依次获得1、2、3的值
```

内置转换类型函数

函数	含义	实例	结果
int(x)	转为不大于x的整型	int('123') int(1.5)	123 1
float(x)	转为实型	float('12.3') float(2)	12.3 2.0
str(x)	将数值x转换成字符串	str(3) str(True)	'3' 'True'
bool(x)	转为布尔值 非0为True,0为False	bool('12.3') bool(0) bool('0')	True False True
eval(x_1,x_2,…,x_n)	计算字符串x的值	eval('3+4') eval('True')	7 True
complex(x_1[,x_2])	转换为复数	complex(1,3)	(1+3j)

```
In [1]: x=12; y=3.14

In [2]: print(type(x),type(y))
<class 'int'> <class 'float'>

In [3]: type(x)
Out[3]: int

In [4]: type(y)
Out[4]: float
```

数学库 math

- import math　　#调用时用math.sin(1.57)
- from math import *　　#调用时直接用sin(1.57)，可直接调用所有数学函数
- from math import sin　　#只能调用指定的sin函数

函数	含义	实例	结果
pi、e	数学常数	pi e	3.1415926535897793 2.718281828459045
sin(x)、cos(x)、tan(x)	正弦、余弦、正切	sin(1.57) cos(0)	0.999999 1.0
exp(x)	以e为底的幂	exp(3)	20.085536923187668
log(x)、log10(x)	自然对数，10为底的对数	log(2.718281828459045) log10(10)	1.0 1.0
sqrt(x)	x的平方根	sqrt(2)	1.4142135623730951

◆ 字符串索引：可正向和反向索引

字符串	w	e	l	c	o	m	e
正向索引编号	0	1	2	3	4	5	6
反向索引编号	-7	-6	-5	-4	-3	-2	-1

◆ 字符串的切片

st[m:n:k]表示从第m个到第n-1个，间隔步长为k

st[:n] 表示从第0个到第n-1个

st[m:] 表示从第m个到最后

st[:] 表示取所有字符

st[::-1] 逆序取所有字符

st[::-2] 逆序取所有偶数索引号上的字符

```
>>> st='welcome'
>>> st[0:2]
'we'
>>> st[-4:-1]
'com'
>>> st[3:7]
'come'
>>> st[:3]
'wel'
>>> st[-4:]
'come'
```

```
>>> st[1:6:3]
'eo'
>>> st[::-2]
'eolw'
>>> st[::-1]
'emoclew'
>>> st[::2]
'wloe'
```

内置字符串处理函数

函数	描述	实例	结果
len(s)	返回字符串长度	len('123456')	6
chr(n)	返回n对应ASCII码字符	chr(65)	'A'
ord(s)	返回字符s的ASCII码值	ord('a')	97

字符串内置方法　　假设st='abc defg bc'

方法	描述	实例	结果
upper()、lower()	转换字母为大写或小写	st.upper()	'ABC DEFG BC'
index(s)	返回第1次出现子串s的位置	st.index('bc')	1
replace(old,new)	用new子串替换old子串	st.replace("bc",'123')	'a123 defg 123'
split([s])	按指定分隔符将字符串分成子串序列	st.split(' ')	['abc', 'defg', 'bc']
isnumeric()	字符串是否全为数字	st.isnumeric()	False

注意：以上方法都不改变字符串本身的内容！！！

关系运算

关系运算符	描述	实例
==	相等	(a == b) 返回 False
!=	不等于	(a != b) 返回 true
>	大于	(a > b) 返回 False
<	小于	(a < b) 返回 true
>=	大于等于	(a >= b) 返回 False
<=	小于等于	(a <= b) 返回 true

```
>>> a=1
>>> b=2
>>> a>b
False
```

```
>>> "a" >"A"
True
>>>1.2-1.0==0.2
False
```

逻辑运算

逻辑运算符	描述	实例
not	取反	not True
and	与	True and True
or	或	False or False

注意：1<x<10　等同于　x>1　and　x<10

成员运算

成员运算符	描述	实例
in	是否在指定的序列中	list = [1, 2, 3, 4, 5] If a in list:
not in	是否不在指定的序列中	If a not in list:

运算优先级

优先级	运算符	描述
1	**	幂运算
2	~ +、-	按位取反 正、负
3	*、/ %、//	乘，除 取余、整除
4	+、-	加法、减法
5	>>、<<	右移，左移
6	&	按位与
7	^、|	按位异或、按位或

优先级	运算符	描述
8	<=、<、>、>= ==、!=	比较
9	=、%=、/=、//= -=、+=、*=、**=	赋值 复合赋值
10	is、is not	身份运算
11	In、not in	成员运算
12	not、and、or	逻辑运算

随机数控制

- import random
- x=random.random()　　返回 [0.0, 1.0) 之间的浮点数
- y=random.randint（a,b）　返回 [a, b] 之间的整数（包括起始与终止值）

```
#每次运行产生的随机数序列是不相同的
import  random
random.seed()
for i in range(10):
    print(random.random())
```

random.seed(n)　　# n可以是整数、小数或字符串，每次产生的序列不变

输入控制

◆ **input（）函数**

<变量>=input（“提示字符串”）

返回的输入结果是**字符串**

例如：x=input('x=')　　# x保存的是字符串

x=float(input ('x='))　# x保存的是转换后的浮点数，若输入字符则报错

```
>>> x=input("请输入x的值=")
请输入x的值=3
>>> x
'3'
```

```
>>> x=float(input('x='))
x=9
>>> x
9.0
```

eval函数：可读入多个数据，也可以输入表达式

```
In [1]: a,b=eval(input('please input number a and b:'))

please input number a and b:1,2

In [2]: a,b
Out[2]: (1, 2)
```

```
In [5]: x,s=eval(input('please input number x and string s: '))

please input number x and string s: 3,'abc'

In [6]: x,s
Out[6]: (3, 'abc')
```

输入字符串必须加引号

```
In [15]: x,y=3,5

In [16]: i=eval(input("输入表达式:"))

输入表达式:x+y

In [17]: i
Out[17]: 8
```

格式化控制输出

（1）用 % 控制

格式符常用形式：% [m][.n]类型符

m为输出最小宽度；n为小数位数；类型符为对应输出数据类型

格式符	%d	%c	%s	%f或%F	%e或%E
类型符	d	c	s	f或F	e或E
对应输出数据类型	整数	单个字符	字符串	浮点数	指数形式

已知 x1=-1.2和x2=-2.5，输出语句：

```
print("x1=%8.3f\nx2=%8.3f"%(x1,x2))        # 输出宽度8位，小数位数3位
```

x1=∪∪-1.200　　# ∪表示空一格

x2=∪∪-2.500

格式化控制输出

占位符即“{ }”也称为“槽”

（2）format方法

形式：**格式字符串.format（输出元组序列）**

```
print('{<参数序号>: <格式控制标记>}'.format(参数1[，参数2),...])
```

<格式控制标记>用来控制参数显示时的格式，包括：**<填充><对齐><宽度>,<.精度><类型>**

:	<填充>	<对齐>	<宽度>	,	<.精度>	<类别>
	用于填充的单个字符	< 左对齐 > 右对齐 ^ 居中对齐	槽的设定输出宽度	数字的千位分隔符 适用于整数和浮点数	浮点数小数部分的精度或字符串的最大输出长度	整数类型 B,c,d,o,x,X 浮点数类型 e,E,f,%

按指定格式输出

- ‘b’ ：输出二进制数字
- ‘c’ ：输出字符
- ‘d’ ：输出十进制整数
- ‘o’ ：输出八进制
- ‘x’ ：输出十六进制
- ‘e’ ：用科学计数法输出
- ‘g’ ：输出小数或指数形式

```
>>> print('{0:b}'.format(97))
1100001
>>> print('{0:c}'.format(97))
a
>>> print('{0:d}'.format(97))
97
>>> print('{0:e}'.format(97))
9.700000e+01
```

```
>>> print('{:g}'.format(12.3))
12.3
```

```
>>> print('{:%}'.format(123))
12300.000000%
```

F 表达式

f-string以f或F开头，后面跟着一个字符串字面量，字符串中可以使用大括号{}来引用变量或表达式

```
name = "Alice"
print(f"Hello, {name}!") # 输出: Hello, Alice!

x = 5
y = 10
print(f"The sum of {x} and {y} is {x + y}.")
# 输出: The sum of 5 and 10 is 15.
```

总结：设计函数的要点　重要

- 给函数起个好名字
- 函数的逻辑实现，需要什么数据？ 定义为对应的形参
- 根据传递的参数，编码实现逻辑
- 逻辑处理的结果，通过return 语句返回

函数的调用

- 格式：**函数名(实在参数表)**
- 调用函数时，( )内的参数被称为**实在参数**，简称**实参**，对应于定义函数时的形参
- **参数传递**：实参与形参**一一对应**且**类型兼容**原则
- **类型兼容**：即当实参传递给形参时，可转换为逻辑需要的数据类型，保证函数运行逻辑不受影响。
- 有 “return 表达式”语句的函数，调用时可用

  变量=函数(实参表)　　否则用　　函数(实参表)

总结—形参分类　四种类型

1. 位置参数：（必选参数），必须给对应的实参
2. 默认值参数（调用时可以不提供对应的实参）
3. 元组可变长参数，参数前有一个*，搜集实参，形成元组
4. 字典可变长参数，参数前有两个*，收集参数，形成字典。（要求实参格式：　标识符=值）

多种形参共存时，顺序该如何排列

1. 如果只有：位置参数、默认值参数，则位置参数在前，默认值参数在后
2. 如果元组变长参数（*参数）与位置、默认值参数共存，处于*参数之前的位置参数，默认值参数，必须遵循位置参数在前，默认值参数在后。*参数后面的位置参数、默认值参数，次序可随意
3. 若元组变长参数、位置参数、默认值、字典变长参数，则字典变长参数必须放最后，其他参数遵循规则2。

- *参数后面的参数，实参必须使用形参关键字方式
- 推荐规则：位置、默认值、元组变长、字典变长

lambda函数

逻辑特别简单的函数，可使用lambda表达式

格式：

- **函数名 = lambda 形参表 : 表达式**
- 函数的调用方式：函数名(实参表)

变量的作用域

- 变量的有效使用范围，称为作用域
- 函数中定义的变量，只在函数内有效，称局部变量
- 主函数里定义的变量，称全局变量

```
def fun(a,b):    a,b,x 局部变量
    x=a+b
    return x     m,n,y 全局变量
m,n=2,3
y=fun(m,n)
print(y)
```

函数中使用全局变量 用global声明

```
x=1
def fx():
    global x
    x=1000
    print("x=",x)

fx()
print ("全局的x=", x)
```

全局的

引用全局变量

执行结果
x= 1000
全局的x=1000

递归问题

- **递归问题的例子：**

求x^n，　$x^0=1$, $x^n=x*x^{n-1}$

求阶乘：n!=n*(n-1)!　　1!=1

- **特点：**

1. **一个大问题，被分解为可用自身规模更小的问题**
2. **递归问题要有一个终止条件，例如上述$x^0=1$。**

递归反应的思维方式：

- **大问题分解为相同类型的小问题，直至最小化（终止），小问题解决了，大问题也就解决了**

3. 列表及其元素的删除

- **del运算符：**　　**del listN[index]或del listN[start : end：]**

已知lt=[1,3,5,7,9]，del lt[3]　#删除第3项，结果为[1, 3, 5, 9]；del lt　#删除列表

- **pop方法：**　**listN.pop(index)**　#index默认是-1

例如：lt.pop(2)　#弹出第2项5，结果为[1, 3, 7, 9]

```
a=[1,2,3,4,5,6,7,8,9]
while a!=[]:
    print(a.pop(),end=" ")  #列表逆序输出
```

```
9 8 7 6 5 4 3 2 1
```

- **Remove方法：**　**listN.remove(列表值或listN[index])**

例如：lt=[10,20,30,10,25,20,30]　**注意：若有重复项，总是删除第1项**

lt.remove(20) 或者**lt.remove(lt[-2])**　#结果都是[10, 30, 10, 25, 20, 30]

- **clear()：删除列表所有元素**

例如：　lt.clear()　#lt的结果为空列表[]

4. 列表的切片

ls[2:4]

15	16.2	8.4	7.6	12.2

- 切片适用于**列表**、元组、字符串、range等类型
- 功能：
  - **截取**列表中的部分元素，得到一个新列表
  - 通过切片**修改或删除**列表中部分元素
  - 通过切片为列表对象**增加**元素
- 形式：ls[m:n:step]，**左闭右开**
  - m: 切片开始位置，默认为0
  - n: 切片截止位置（**不包含**），默认为列表长度
  - step：步长，默认为1
  - 全部省略时取整个列表

5. 列表运算符：　+、* 和 in

- **+ 实现增加元素，返回新列表**

```
fibs=[0,1,1,2,3]
fibs+[5,8]        # 显示：[0, 1, 1, 2, 3, 5, 8]，fibs列表本身并没改变
fibs+=[6]         # fibs列表变为 [0, 1, 1, 2, 3, 6]
```

- *** 实现序列重复，返回新列表**

```
fibs*2            # 显示 [0, 1, 1, 2, 3, 6, 0, 1, 1, 2, 3, 6]，fibs没变
fibs*=0           # fibs变为空列表[]
```

- **in 用于测试列表中是否包含某个元素**

```
fibs=[0,1,1,2,3,5,8]
3 in fibs         # True
'3' in fibs       # False
```

5.1.3 列表对象方法 — append

◆ **列表.append(object)　　#向列表尾部追加一个元素**

x.append(6,7)　#报错：　append() takes exactly one argument (2 given)

x.append([6,7]) # 追加一个列表元素[1, 5, 8, 6, [6, 7]]

◆ **思考**：append方法与列表的 + 运算有何不同?

+ 只能连接一个列表

+ 可能会改变内存地址，append 则不会

例5.3　计算斐波那契数列前10项

```
fib=[0,1]
for i in range(2,10):    #从2开始
    fib.append(fib[i-1]+fib[i-2])
    # fib +=[ fib[i-1]+fib[i-2] ]
```

```
    fib[i] =fib[i-1]+fib[i-2]
IndexError: list assignment index out of range
```

```
#修订如下：
fib=[0,1]+[0]*8
for i in range(2,10):
    fib[i] =fib[i-1]+fib[i-2]
```

列表对象常用方法

设x=[1,5,8,6]

x.insert(index,object)　#将元素插入在指定索引位置

x.insert(0,2)　　# 将2插在索引为0处：　[2, 1, 5, 8, 6]

x.extend(iterable)　　#将序列中的所有元素添加在末尾

x.extend([11,12,13])　　# [2, 1, 5, 8, 6, 11, 12, 13]

x.sort()　#列表元素排列，降序指定：reverse=True，列表中元素是可比较的

L=['abc',1,3,18.9]; L.sort()×

L.sort(key=str)√　　# 按字符串顺序，数值自动转换成字符串

x.reverse()　　#对列表元素逆序排列

列表对象方法 — index

◆ **功能**：返回指定元素在列表中首次出现的位置

◆ **格式**：**列表.index(元素)**

```
L1=[1,3,6,3,3,7,8,1,1,1]
pos1=L1.index(3)
print(pos1)              # 输出3这个的元素首次出现的索引号1
```

不存在则抛出异常

```
pos2=L1.index(0)         # 注意列表中没有0这个元素
print(pos2)              # ValueError: 0 is not in list
```

#改进: 用选择结构给出提示，避免出现异常

```
if 0 in L1:
    print(L1.index(0))
else:
    print("不存在")
```

列表对象方法 — count

◆ 对指定的元素计数，即该元素在列表中出现的次数

◆ 格式：**列表.count(元素)**

```
L=['hellen','stery','antongni','hellen']
n=L.count('hellen')
print(n)                 # 结果：2
```

5.1.4 操作列表的内置函数

- **sum:** 返回列表中所有元素之和
- **len:** 返回列表的元素个数
- **max，min:** 返回列表中的最大和最小值
- **map:** 将指定函数作用于列表中的每个元素
- **zip:** 将**多个列表的对应元素整合为元组**
- **enumerate:** 将**一个列表的元素与其索引组成元组**
- **filter:** 按指定条件过滤列表的元素

5.1.6 元组（tuple）

- 定义： 元组名=（数据1，数据2，……）

  例：t1=(10,20,30)；t2=tuple([1,2,3])
- 元素类型不受限制

  t3=([11, 22], 33, " 88 ")
- 只有一个数的元组（10，）
- 使用： 元组[下标]
- 特点：**元组元素不可变更**， 系统开销比列表小

```
t3[1]=55×
t3[0].append(-55)
print(t3)        #输出结果：([11, 22, -55], 33, " 88")
```


5.2 字典与集合

5.2.1 字典的创建

字典：以键值对形式存储数据，键唯一，值类型可任意

特点：字典中的对象是无序的（不可排序）

1. 逐项创建 字典名={键：值，……}

例： Dcountry={'中国':'北京','美国':'华盛顿','德国':'柏林'}

2. 用dict()转换

```
data=[[“中国”，“北京”],[“英国”，“伦敦”],[“美国”，“华盛顿”]]
Dcountry=dict(data)   #结果为 {'中国': '北京', '英国': '伦敦', '美国': '华盛顿'}
```


字典属性及其访问

- len(d) 返回 d 中“键-值”对的数量，即表项数目
- d.keys() 返回所有键
- d.values() 返回所有的值
- d.items() 返回所有键值对
- d.get(key, default=None) 返回字典键 key 的值
- d[k] 返回键 k 对应的值,K不存在报错
- d[k] = v 将值 v 与键 k对应
- del d[k] 删除键为 k 的项
- k in d 判断 k 是否是 d 中的一个键

5.2.2 集合运算

设有如下集合:

s1=set((1,3,7,8))

s2=set([2,8,10])

运算符	含义	示例	结果
-	差集	s1-s2	{1,3,7}
&	交集	s1&s2	{8}
|	并集	s1|s2	{1,2,3,7,8,10}
^	对称差集 (两个集合中不重复的元素集合)	s1^s2	{1,2,3,7,10}

集合对象的方法

方 法	含 义
s.add(x)	增加新元素x
s.update(x)	将序列x中的每一项加入集合s
s.remove(x) s.discard(x)	从集合s中移除元素x，若x不存在，报错 从集合s中移除元素x，若x不存在，不报错
s.clear(x)	清除集合
s.difference(x)	差集
s.intersection(x)	交集
s.issubset(s1)	s1是否是s的子集
s.union(x)	并集
s.symmetric_difference(x)	对称差集

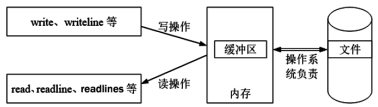
文件访问流程



一般来说，访问数据文件的程序由三部分组成：

首先要创建流对象，然后通过流对象进行读写等操作，最后要关闭流对象。

这三步相当于打开文件，读写文件，关闭文件。



读操作，即输入，将数据从文件读入到变量供程序使用

写操作，即输出，将数据写入文件。

6.2.1 文本文件的打开与关闭

1. 打开文件

fp = open(filename, mode='r', encoding='utf-8')

fp文件对象的属性

属性	描述
fp.closed	如果文件已被关闭返回True，否则返回Flase。
fp.mode	返回被打开文件的访问模式。
fp.name	返回文件的名称。

```
In [19]: fp=open(r"d:\data\drunks.txt", encoding="utf-8")
In [20]: type(fp)
Out[20]: _io.TextIOWrapper
```

```
fp=open("d:\\data\\gc-4-1.py","r")
print(fp.name)
print(fp.closed)
print(fp.mode)
```

```
In [1]: runfile('D:/Python untitled0.py', wdir='D:/Py')
d:\data\gc-4-1.py
False
r
```

文件的读写模式

```
fp1 = open("d:\\abc.txt" ,"r")          #以读模式打开
fp2 = open(r"D:\jsjjc\student.txt", "w")    #以写模式打开
```

Mode	作用	说明
r	读模式（默认）	文件不存在则异常
w	写模式	文件不存在则创建；文件存在则清空内容
x	写模式	创建新文件，文件存在则异常
a	追加模式	文件存在则追加；文件不存在则创建
b	二进制模式	可与其他模式组合使用
t	文本模式（默认）	可与其他模式组合使用
+	读/写模式	打开文件进行更新，可以其他模式组合使用

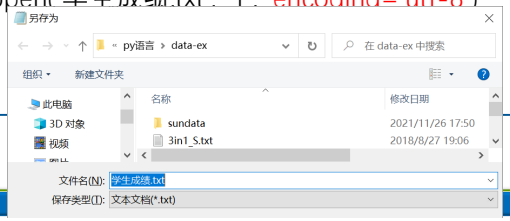
- 模式间合并，如'rb'，二进制读

补充—文件的编码问题

- 有汉字的文件，经常用 utf-8 编码，打开时需要指定编码

例子

f=open('学生成绩.txt', 'r', encoding='utf-8')



2. 文件关闭：close()方法

- 通常数据是被送到缓冲区，关闭文件时才将缓冲区中数据全部写入文件
- 关闭文件对象：文件操作完以后一定要关闭

文件对象.close()

例：fp.close()

6.2.2 文件的读写

1. 读文件

- 前提 f=open('文件名','r', encoding='utf-8')
- 常用四种方式：

1. 将文件看作一个列表，for line in f: line是一个字符串
2. read方法：一次读取全部文件内容，整个文件形成一个字符串 txt=f.read()
3. readline方法：一次读取文件中的一行，一行形成一个字符串 line=f.readline()
4. readlines方法：一次读取文件中的所有行,以行为单位形成列表，即列表中的每个元素，是文件的

写文件

- 将程序计算结果(内存中的内容,即变量中的值)，保存到文件
- 文件要以'w'、'x'、'a'模式打开

f=open('文件名','w')

文件已经存在时，内容清零，不存在时，则创建

- 常用的两种写文件方法
- f.write(x): 变量x写入文件，x必须是str类型,不是字符串，则用str函数转
- f.writelines(x)：x是列表或字符串，x的元素必须是str类型
- 总结：字符串、字符串、字符串

1、write方法

形式

fp.write()

说明:

1、一次只能写一个字符串

若写入多个数据，只能使用多个语句:

例如:

fp.write('1850001',"张杰",'90') 结果

要写成:

fp.write('1850001')

fp.write('张杰')

1850001张杰90
行不行?

write续

2、 参数只能是字符串

fp.write("90") ✓

fp.write(90) ✗

3、写数据时不会自动换行，若要换行，加入'\n'

4、字符串写入文件不会自动添加双引号

可以 用字符串的+运算，如

fp.write('1850001\t'+'张杰\t'+'90\n')

2、writelines方法

- 文件对象.writelines(元素是字符串的列表)

  fp.writelines(字符串构成的列表)
- 将存放在列表中的多行数据一次性写入文件
- 若要换行，在列表元素的末尾添

```
fp=open(r"D:\python\Sushi.txt","w")
lines=["竹外桃花三两枝，\n","春江水暖鸭先知。\n","蒌蒿满地芦芽短，\n","正是河豚欲上时。\n"]
fp.writelines(lines)
fp.close()
```

Sushi.txt - 记事...
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
竹外桃花三两枝，
春江水暖鸭先知。
蒌蒿满地芦芽短，
正是河豚欲上时。

```
#改用with
with open(r"poem1.txt", 'w') as f:
    for line in lines:
        f.write(line+"\n")
f.close()
```

将列表写入文件，不分行，若要

模式与操作

r模式打开，不能写，w模式打开，不能读

```
f=open(r'd:\python\data-ex\file3.txt','w')
lines=['竹外桃花三两枝\n','春江水暖鸭先知\n','蒌蒿满地芦芽短\n','正是河豚欲上时\n']
f.writelines(lines)
all=f.read()
print(all)
f.close()
```

思考：

- 增加两句读文件的语句，结果是什么？为什么？
- 怎么才能读出来？
- ✓ 关闭文件后重新以读方式打开

CSV文件

- 文本格式，列间逗号为分隔符
- 扩展名csv（Comma Separated Value）
- 用与文本文件一样的方法处理

1950001	张三	90	95	80
1950002	李四	88	70	92
1950003	王五	77	90	82

例：如上，文件中记录一组学生3们课程的分数，读取文件内容，求所有分数的平均分


```
file='d:/python/data-ex/scores.csv'
f=open(file,'r')
total=0
count=0
lines=f.readlines()
for line in lines:
    temp=line.split(',')  # 因为是 csv
    for i in range(2,5):
        total=total+int(temp[i])
        count=count+1
average=total/count
```


jieba分词

- jieba 是 python 的中文分词工具，它提供了多种分词模式，把中文文本中的各种词组的组合分解出来
- pip install jieba　　#安装jieba库
  pip install jieba -i https://mirrors.aliyun.com/pypi/simple/
- jieba库的中文分词模式有三种：精简模式、全模式、搜索引擎模式

1. 精简模式

将句子进行最精简的分词，适合文本分析。使用lcut方法实现，其返回值是列表。

```
import jieba
lst =jieba.lcut('小明本科毕业于同济大学，后在耶鲁大学深造')
print(lst)    #结果：['小明', '本科毕业', '于', '同济大学', '，', '后', '在', '耶鲁大学', '深造']
```

2. 全模式

把句子中所有可以成词的词语都扫描出来，速度非常快，但其分词得到的结果有可能会有歧义。

```
import jieba
lst=jieba.lcut('小明本科毕业于同济大学，后在耶鲁大学深造',cut_all=True)
print(lst)
#显示结果：['小', '明', '本科', '本科毕业', '毕业', '于', '同济', '同济大学', '大学', '，', '后', '在', '耶鲁', '耶鲁大学', '大学', '深造']
```

3. 搜索引擎模式

在精确模式的基础上，对长词再次切分，适合于搜索引擎分词

```
import jieba
list1=jieba.lcut_for_search('小明本科毕业于同济大学，后在耶鲁大学深造')
print(list1)
#显示结果：['小明', '本科', '毕业', '本科毕业', '于', '同济', '大学', '同济大学', '，', '后', '在', '耶鲁', '大学', '耶鲁大学', '深造']
```

4. 文本关键词抽取

extract_tags(待分析文本, topK=20, withWeight=False, allowPOS=( ))

- topK 参数返回前n个权重最高的词，默认20 个
- withWeight 指定是否计算关键字的权重
- allowPOS 仅包括指定词性的词，默认值为空，取值：（'ns', 'n', 'vn', 'v '）

对应提取地名、名词、动名词、动词

```
import jieba.analyse as analyse
file = open(r"d:\data\红楼梦.txt", "r", encoding='utf-8')
txt=file.read()
lists=analyse.extract_tags(txt, topK=5, withWeight=False, allowPOS=( ) )
print(lists)                # 输出结果：['宝玉', '袭人', '王夫人', '凤姐', '丫头']
```

4.1 图像基本处理

Image是PIL中最重要的类，可以用来实现图像处理的所有基本功能

- Image.open(filename,mode) 打开一张图像，

返回一个Image对象，该对象有size,format,mode等属性

- size表示图像的宽度和高度(像素表示)
- format表示图像的格式,常见的包括JPEG,PNG等格式
- mode表示图像的模式，定义了像素类型还有图像深度等，常见的有RGB,HSV等。

例如：

```
from PIL import Image
im = Image.open("d:/images/hm.jpg","r")
print(im.size,im.format,im.mode)     #显示图像的大小，格式和模式
im.show()        #显示图像
```

```
(500, 380) JPEG RGB
```

1．图像打开、显示、转换和保存

（1）打开图像： Image.open(filename)

（2）显示图像： im.show()

（3）保存图像： Image. save(filename,format)

```
im = Image.open(r"digit.jpg")
im.save（“digit.bmp”，“bmp”）      #存成bmp格式
```


（4）图像转换　　图像对象.convert(格式)

例如：

```
from PIL import Image                          # 导入PIL库的Image模块
im = Image.open(r"D:\images\Tulips.jpg") # 打开图像文件
im_L = im.convert("L")
im_L.show()
im_rgb = im_L.convert("RGB")
im_rgb.show()
```

convert方法可以改变图像的mode，一般是在‘RGB’（真彩图）、 ‘L’（灰度图）、‘CMYK’（印刷模式)之间转换。

注意：前面将图像转化为灰度图，再从灰度图转化为真彩图，但是恢复不了原来的真彩色

。

PIL支持9种图像模式：　1，L，P，RGB，RGBA，CMYK，YCbCr, I，F

- 模式“1” 二值图像，每个像素用8个bit表示，0表示黑，255表示白
- 模式“L” 为灰色图像，每个像素用8个bit表示，0~255对应256个灰度级别
- 模式“P” 为8位彩色图像，每个像素用8个bit表示，其对应的彩色值是按照调色板查询出来的。
- 模式“RGBA” 为32位彩色图像

  每个像素用32个bit表示，其中24bit表示RGB三个通道，另外8bit表示alpha通道，控制透明
- 模式“CMYK”为32位彩色图像，每个像素用32个bit表示，是印刷的四分色模式
- **模式“YCbCr”**为24位彩色图像

  每个像素用24个bit表示，Y是亮度，Cb：蓝色色度分量，Cr：红色色度分量
- 模式“I”为32位整型灰色图像，
- 模式“F”为32位浮点灰色图像，每个像素用32个bit表示，0~255对应256个灰度级别

（5）像素点处理

图像对象.getpixel((x,y))　　#获取指定位置像素点的颜色值

```
from PIL import Image                          # 导入PIL库的Image模块
im = Image.open(r"D:\images\Tulips.jpg")   # 打开图像文件
print(im.getpixel( (100,80) ))                 # 返回指定位置的RGB值(156, 204, 253)
```

图像对象. putpixel(self, (x,y) , color) #修改指定像素点的颜色

```
from PIL import Image
im = Image.open(r'd:\images\hm.jpg')
width, height = im.size
for x in range(width):
    for y in range(height):
        a=im.getpixel((x,y))     #获取了每个像素点的RGB值（元组）
        r,g,b=a                  #先解包获取各个颜色值
        b=(r*10,g+b,b-50)        #每个颜色处理后重新放到元组中
        im.putpixel( (x, y), b ) #修改每个像素点的颜色值
im.show()
```

(6) 更改像素点的值

图像对象.point(lut,mode=None)

```
# 对图像的每个像素点进行改变，lut：look up table 查找表
from PIL import Image                          # 导入PIL库的Image模块
im = Image.open(r"D:\images\Tulips.jpg")       # 打开图像文件
im1=im.point(lambda x: 255 if x > 127 else 0)  #lambda的作用：使图像三个通道的值都改
im1.show()
```


图像模式转换，去除背景灰度



```
from PIL import Image
im = Image.open(r"d:\images\digit.jpg")
im.show()
im = im.convert("L")                              # 转换为灰度8位图像
im = im.point(lambda x: 255 if x > 196 else 0)    # 像素点处理
    #代表数字的图像的像素值是0
im1 = im.convert("1")             #用1位表示颜色，只有黑和白， 取值True或False
im1.save("sample_1.bmp")      # 转存为位图
im1.show()
```


2. 图像的变换

- transpose(method)　　#图像翻转或者旋转

```
im_new = im.transpose(Image.ROTATE_180)    #画布旋转
im_new.show()
```

method是transpose的参数，表示选择什么样的翻转或者旋转方式，可以选择的值有:

- Image.FLIP_LEFT_RIGHT　表示将图像左右翻转
- Image.FLIP_TOP_BOTTOM　表示将图像上下翻转
- Image.ROTATE_90　表示将图像逆时针旋转90°
- Image.ROTATE_180　表示将图像逆时针旋转180°
- Image.ROTATE_270　表示将图像逆时针旋转270°
- Image.TRANSPOSE　表示将图像进行转置(相当于顺时针旋转90°)
- Image.TRANSVERSE　表示将图像进行转置，再水平翻转

3. 色彩通道的分离和合并

- RGB图像每个像素都是RGB三个分量
- 分离：把所有像素的RGB分量分开，生成三个列表
- 合并：分离的逆操作，通过3个颜色通道合并成新图像
- split()　颜色通道分离

```
r,g,b = im.split()
r.show(); g.show(); b.show()
```

split()方法可以使原来图像的各个通道分离, 对RGB图像，可分离成3个通道

- merge(mode,channels)　颜色通道合并

```
im_merge = Image.merge("RGB",[b,r,g])
im_merge.show()
```

merge方法和split方法相对的，将多个单一通道的序列合并起来，组成一个多通道的图像

mode是合并之后图像的模式，比如"RGB"

channels是多个单一通道组成的序列。

改变图像大小

- resize(size,resample)　#改变图像大小

```
im_resize = im.resize((200,200))
im_resize.show()
im_resize_box = im.resize((100,100)，Image.NEAREST)　#指定重采样方法
im_resize_box.show()
```

  - size是转换之后的大小，二元组：（width, height）
  - resample是重新采样使用的方法
    - PIL.Image.BICUBIC　(4*4的三次样条插值)
    - PIL.Image.LANCZOS　(Lanczos迭代法)
    - PIL.Image.BILINEAR　(2*2的线性插值)
    - PIL.Image.NEAREST　(最近邻插值法)

图像复制—合成

- paste(region,box,mask)　将一个图像粘贴到另一个图像上

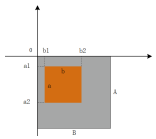
```
im.paste(region,(100,100),None)
im.show()
```

- 将region图像粘贴到左上角为(100,100)的位置。
- region是要粘贴的Image对象,box是要粘贴的位置，可以是2个元素的元组，表示粘贴区域的左上角坐标,也可以是一个4个元素的元组，表示左上角和右下角的坐标。
- 如果是四个元素元组的话,box的size必须要和region的size保持一致

图像的裁切

- crop(box)　裁剪矩形区域

```
im = Image.open("digit.jpg","r")
box = (100,100,200,200)
region = im.crop(box)
region.show()
```



box（b1,a1,b2,a2）

- 在im图像上裁剪一个box矩形区域，然后显示出来
- box是一个包含四个数字的元组(upper_left_x,upper_left_y,lower_right_x,lower_right_y)

即裁剪矩形区域的左上角x,y坐标，右下角的x,y坐标

- 规定图像的最左上角的坐标为原点(0,0)，宽度的方向为x轴，高度的方向为y轴
- crop()返回的仍然是一个Image对象。

4.2 图像的滤波和增强

- filter(filter)　# 滤镜

```
from PIL import Image
im = Image.open("d:\images\hm.jpg","r")
from PIL import ImageFilter
im_blur = im.filter(ImageFilter.BLUR)
im_blur.show()
im_blur.save("blur.jpg")
im_contour = im.filter(ImageFilter.CONTOUR)
im_contour.show()
im_contour.save("contour.jpg")
```

```
BLUR　#变模糊
CONTOUR　#黑白边框素描
DETAIL　#变清晰
EDGE_ENHANCE　#边缘明显
EMBOSS　#灰色浮雕
SHARPEN　#锐化
FIND_EDGES　#寻找边缘信息
```

- filter是过滤器函数，在PIL.ImageFilter函数中定义了大量内置的filter函数，比如BLUR(模糊操作)，GaussianBlur(高斯模糊)，MedianFilter(中值过滤器)，FIND_EDGES(查找边缘)等。

七、图像的滤波和增强

功能	使用类	例子
色彩增强	ImageEnhance.Color	en=ImageEnhance.Color(im) im1=en.enhance(2) im1.show()

亮度	ImageEnhance.Brightness
	# Brightness是一个派生类
		en=ImageEnhance.Brightness(im)
		im1=en.enhance(2) # 增强一倍
		im1.show()

对比度	ImageEnhance.Contrast	en=ImageEnhance.Contrast(im) im1=en.enhance(2) im1.show()
锐度增强	ImageEnhance.Sharpness	en=ImageEnhance.Sharpness(im) im1=en.enhance(2) im1.show()

7.1 基本概念

➢对象：现实世界的实体，每个对象都有自己的特征和行为

具体的概念，如某个人，某辆汽车

➢类：对一组对象共有属性特征和行为的抽象，如人类，汽车等

➢数据描述：数据名称、类型

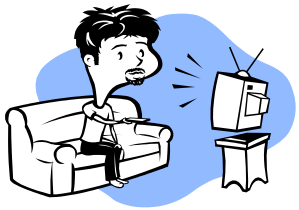
➢行为描述：有哪些行为，如何实现

➢抽象：一个类是一组对象的抽象，抽取出用得上的共同特征

完成必须完成的动作。

7.1.2 类的特点

1.封装性：对象的行为和属性是一个有机的整体，将对象的状态和行为打包在一个软件构件中。



封装性：将对象的所有成员集成在一起（暗箱）

➢ 隐藏：保护内部细节，类内的某些成员可以被保护起来

➢ 作用：具有较好的独立性，防止外部程序破坏类的内部数据，便于程序维护

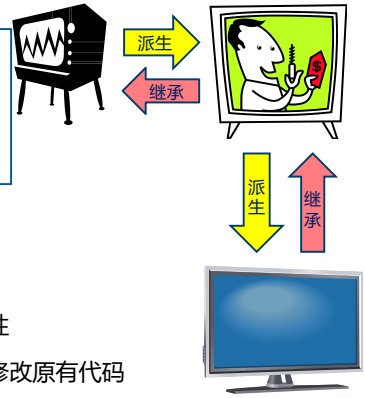
2. 继承性

◆ 电视机：黑白→彩色→高清

其中声音和图像的功能一直延续下来

◆ 后一代“**继承**”前一代的功能并有所发展

◆ 旧电视**派生**出新电视，新电视**继承**了旧电视



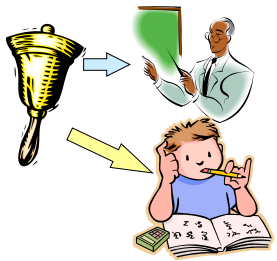
基类（父类）→ 派生类（子类）

➢ 子类拥有父类的特性，又可增加子类的新特性

➢ 设计程序时，只需考虑添加新功能的代码或修改原有代码

3. 多态性

◆ **同一操作作用于不同类的对象，将产生不同的执行结果**，即不同类的对象收到相同的消息时，得到不同的结果。






```
教师_铃声 ()
{ 打开电脑
  讲课
  ...
}
```

```
学生_铃声 ()
{ 听讲
  记笔记
  ...
}
```

```
其他人员_铃声 ()
{
}
```

类Student解释

```
class Student:
    # SetValue方法用于定义该类对象的数据属性
    def SetValue(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    # Disp方法用于输出该类对象的数据属性值
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
p=Student()                              # 创建Student对象
p.SetValue("2020001", "张三", 88)  # 对象初始化
p.Disp()
```

类名 Student

①类Student中定义了SetValue和Disp函数，是类的函数，或叫方法

② p=Student()

创建Student类的一个实例，并且赋值给p（绑定）。

类成员的属性

成员	形式	权限
私有成员	__X	两个下划线，在类的内部可以访问

```
class Student:
    # SetValue方法用于定义该类对象的数据属性
    def SetValue(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    # Disp方法用于输出该类对象的数据属性值
p=Student()                              # 创建Student对象
p.SetValue("2020001", "张三", 88)  # 对象初始化
print(p.No)
```

```
class Student:
    # SetValue方法用于定义该类对象的数据属性
    def SetValue(self,a,b,c):
        self.__No=a;self.__Name=b;  self.__Score=c
    # Disp方法用于输出该类对象的数据属性值
    def disp(self):
        print(self.__No)
p=Student()                              # 创建Student对象
p.SetValue("2020001", "张三", 88)  # 对象初始化
print(p.__No)
p.disp()
```

类的定义语法格式

① **class** 类名(父类名): （父类缺省为object）

② 类体前的字符串说明类的帮助信息，可以通过**类名.__doc__**获得

③ **类体包括类的方法和数据属性，方法和数据都称为类的成员**

④ 构造函数**__init__(self,参数)**是一种特殊的方法，创建实例时被调用，前后2个下划线

⑤ 类体中其他方法可以通过实例调用，格式为： 对象.xxx

⑥ 类内的方法**第一个参数都是self，self是对象自身的引用**

⑦ **创建实例对象**，使用**类名()**或者**类名(初始化参数)**来创建，调用构造函数

⑧ **实例调用方法**：**实例对象.方法名(实参)**， 如：p.SetValue("2020001", "张三", 88)

类的定义语法格式，此页请背诵牢记

```
class 类名:
    def __init__(self, 形参列表): # 构造函数
    # 当一个类中，不编写 __init__函数时，系统自动提供无参的构造，对象实例化时调用构造函数， 类似定义了一个 __init__(self) 的函数
        ...
    def 函数名1(self,形参列表): # 函数1
      ...
    def 函数名n(self,形参列表): # 函数n
```

类的每个函数的定义时，第一个参数，都是self

7.2.3 类的实例化、对象成员的访问

类不能直接调用，必须实例化后

1．类的实例化

变量=类()　　#无参

变量=类(实在参数表)　#有参，取决于__init__函数

变量与实例的关系： 绑定或引用

s=Student()

2．对象成员的访问

数据成员（属性）：对象变量.数据成员名

方法：对象变量.方法()

对象变量.方法(实在参数表)

3. 调用时，定义时的self，在实参表中忽略

```
class Student:
    def __init__(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
#p=Student()
p=Student("1900001", "王涛", 82)
p2=p
p2.name='李蒙'
p.Disp()
```


7.3 继承和派生

1．引例

```
#直接定义学生干部类
class StudentLeader:
    def SetValue(self,a,b,c,d):
        self.No=a
        self.Name=b
        self.Score=c
        self.Duty=d
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
        print(self.Duty)
```

```
#通过派生定义学生干部类
class StudentLeader(Student):
    def new_SetValue(self,a,b,c,d):
        self.SetValue(a,b,c) # 继承自父类，调用格式
        self.Duty=d
    def New_Disp(self):
        self.Disp() # 继承自父类　，
        print(self.Duty)
```

```
sLeader=StudentLeader()
sLeader.new_SetValue("1900001","王涛",82,"班长")
sLeader.New_Disp()
```


2.派生类的定义

class　派生类名(基类名)：

　　'类文档字符串'

　　类体

3. 派生类的构造函数

__init__(派生类构造函数总参数表)：

　　基类名.__init__(基类构造函数参数表)

　　定义派生类对象属性数据成员初始化

派生类的构造函数

父类

```
class Student:
    def __init__(self,a,b,c):
        self.No=a
        self.Name=b
        self.Score=c
    def Disp(self):
        print(self.No)
        print(self.Name)
        print(self.Score)
```

```
#通过派生定义学生干部类
class StudentLeader(Student):
    def __init__(self,a,b,c,d):
        Student.__init__(self,a,b,c)
        self.Duty=d
    def New_Disp(self):
        self.Disp()
        print(self.Duty)
sLeader=StudentLeader("1900001","王涛",82,"班长")
sLeader.New_Disp()
```


7.4 对象迭代器和生成器

7.4.1 迭代器

1．可迭代对象：能够一次返回它的一个成员的对象。任何可以用for循环遍历的对象都是可迭代对象，如**字符串、列表、元组、字典、集合都是可迭代对象**。

2．迭代器：用来从可迭代对象中进行取值的循环方法，必须用__iter__()和__next__()实现

```
class Fibs():
    def __init__(self,max):
        self.max=max
        self.n,self.a,self.b=0,0,1
    def __iter__(self):        #定义__iter__方法
        return self
    def __next__(self):        #定义__next__方法
        if self.n<self.max:
            tmp=self.a
            self.a,self.b=self.b,self.a+self.b
            self.n+=1
            return tmp
        raise StopIteration        # 停止迭代
fib10=Fibs(10)  # 求前10项
for item in fib10:
    print(item,end=" ")
```

3：当一个类，想让其具备迭代能力时，定义2中的函数

我是谁？ 我在哪？ 我在做啥？

7.4 迭代器和生成器

7.4.2 生成器

1．**生成器：** 也属于迭代器，作用于函数（不是类），所以无须写__iter__()和__next__()函数

◆ **生成器函数使用yield关键字返回函数值**

yield与return作用相似，但它能记住函数的状态，**下次调用生成器函数时，它不是从头开始，而是从上次调用中停止的位置开始**。

2．**生成器表达式**

生成器表达式与列表推导式类似，区别是包含在圆括号内

```
square_gen = (x*x for x in range(1,10))
for i in square_gen:
    print(i)
```

```
def Fibs(n):  # 打印前n项
    k=0  # 项数索引
    a,b=0,1
    while k<n:
        value=a
        a,b=b,a+b
        k += 1
        yield value   # 冻住
for item in Fibs(10):
    print(item,end=" ")
```

如果改成return会怎么样?

```
def Fibs(n):
    a,b=0,1
    while a<n:
        value=a
        a,b=b,a+b
        return value
```


7.5 模块、库、包

◆ **模块（module）：即一个.py文件，**每个模块中定义了一些变量、函数等。

- **使用模块的好处：**
  1．提高代码的可维护性
  2．提高代码的可复用性。当一个模块编写完成之后，可以被其他模块引用。
  3．使用模块还可以避免变量名、函数名、类名等命名冲突。

◆ **包（package）就是一个内含模块或子包的有层次的文件目录结构**

其中一定要有__init__.py模块。

1. 每个包目录中必须含有__init__.py模块，否则Python会将视为普通目录，而非一个包。__init__.py文件内容可以为空，也可以有相应的代码。

2. 一个包目录中还可以存在多个目录，称为子包，组成多层级的包。

7.5 模块、库、包

◆ **库（lib）**：是具有相关功能模块的集合。库是一个抽象概念，任意一个模块或包都可以称为库。

- **标准库** Python自带的库。math、random等模块都是其中的一模块。
- **第三方库** 就是第三方发布的具有特定功能的模块。Jieba、Matplotlib、NumPy、pandas、scikit-learn、TensorFlow、PyTorch、Keras、SciPy等
- **自定义模块** 用户自己可以自行编写模块，然后使用。

类的调用

1. 生成对象，格式

   对象名=类名(实参列表) 对标 __init__() 函数的定义
2. 调用类的函数，格式：

   对象名.函数名(实参列表)
3. 调用类的属性，格式

   对象名.属性
4 类内不同函数间的调用

   self.函数名(实参表)

2.图表的组成

Matplotlib的图表区包含三个层次

◆ **容器层** 由Canvas（画布）、Figure（绘图区）和Axes（坐标系）组成

- **Canvas** 位于容器层的最底部（由系统自动创建）
- **Figure** 则建立在Canvas上面，
- **Axes** 则建立在Figure上面

一个Figure对象内可以包含多个Axes对象，每个Axes对象都是一个独立的坐标系，绘图过程中所有图像都是基于坐标系绘制的。

◆ **辅助显示层** 包括标题、坐标轴、、坐标轴刻度、刻度标签、图例等。

◆ **图像层** 绘制的柱状图、折线图、饼图、散点图等图像。

子绘图区

（1）subplot( )函数 在全局绘图区创建子绘图区并显示。

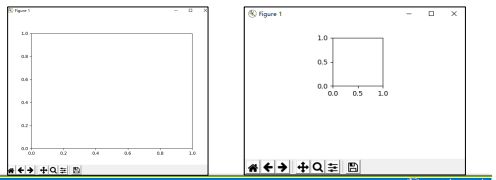
subplot(nrows, ncols, index, **kwargs)

注意：如果nrows、ncols和index这三个数都小于10的话，可以把它们缩写为一个整数，例如subplot(323)和subplot(3,2,3)是相同的。

例如：

```
>>>plt.subplot(1,1,1)
#表示有1行1列，绘制在第1个子绘图区

>>>plt.subplot(232)
#有2行3列共6个子绘图区，只显示第2个
```



子绘图区

Ax是个2行3列的数组，每个元素是axes对象

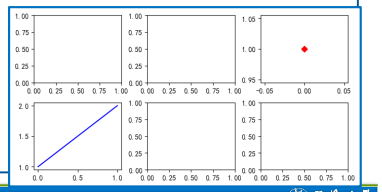
（2）subplots( )函数

创建全局绘图区，然后添加多个子绘图区并显示所有子绘图区

例如：

数组元素指明在那个坐标系

```
import matplotlib.pyplot as plt
fig,ax = plt.subplots(nrows=2,ncols=3,figsize=(8,4),dpi=100)
plt.subplots_adjust(wspace=0.3,hspace=0.3)
ax[0][2].plot((1),'rD')
ax[1][0].plot((1,2),'b')
plt.show( )
```



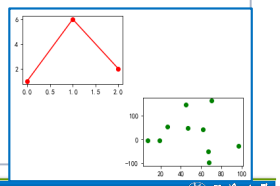
思考：只有一个子图，一行（列）子图，ax是什么?

子绘图区

（3）add_subplot(nrows, ncols, index[, **kwargs])

用于给已有全局绘图区添加子绘图区

```
import matplotlib.pyplot as plt
import numpy as np
fig1=plt.figure( )                  #创建一个全局绘图区
fig1.add_subplot(221)               #添加2行2列共4个子绘图区，只显示第1个
plt.plot((1,6,2),c='r',marker='o')  #在第1个子绘图区绘制红色折线并输出标记
fig1.add_subplot(224)               #指定显示第4个
x=np.random.randint(0,100,10)       #随机生成10个坐标点（x,y）
y=np.random.normal(0,100,10)        #[0,100)之间
plt.scatter(x,y,c='g')              #绘制在第4个绘图区
plt.show( )
```



常用图表绘制函数

函数	描述
plt.plot(x,y,color,linewidth,linestyle,label)	根据x,y数组绘制直线或曲线
plt.boxplot(data,notch,position)	绘制一个箱型图
plt.bar(x, height, width, bottom, *, align='center', **kwargs)	绘制一个条形图（即柱状图）
plt.barh(y, width, height, left, *, align='center', **kwargs)	绘制一个横向条形图
plt.polar(theta,r)	绘制极坐标图
plt.pie(data,explode)	绘制饼图
plt.scatter(x,y)	绘制散点图
plt.hist(x,bins,normed)	绘制直方图
plt.contour(X,Y,Z,N)	绘制等高线

坐标轴设置函数

函 数	描 述
plt.axis( ) 输出：(0.0, 1.0, 0.0, 1.0)	显示当前坐标轴刻度的取值范围
plt.axes( ) 输出：<matplotlib.axes._subplots.AxesSubplot at 0x22f751cb3c8>	显示子绘图区
plt.xlim(xmin=1,xmax=5)	设置X轴刻度取值范围为[1,5]
plt.ylim(ymin=3,ymax=6)	设置Y轴刻度取值范围为[3,6]
plt.text(x,y,s,fontdic,withdash) 如：plt.text(0.5,0.5,'Max')	在指定位置添加注释文本 在两个坐标轴的中间位置输出 “Max”
plt.grid(True/False)	是否显示绘图网格

标签设置函数

函 数	描 述
plt.title("TITLE",loc='right')	设置图表的标题 loc:{'center','left','right'}
plt.xlabel(“水平轴标题”)	设置X轴的标题
plt.ylabel(“垂直轴标题”)	设置Y轴的标题
plt.xticks( ) 如：plt.xticks(np.arange(3), ('Tom', 'Dick', 'Harry'))	设置X轴的刻度范围及显示标签
plt.yticks( ) 如：plt.yticks(np.arange(3), ('1', '2','3'))	设置Y轴的刻度范围及显示标签

解决汉字和负号显示问题

□ 方法1：在代码中添加语句

```
plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus']=False
```

□ 方法2：修改系统配置文件

找到C:\Users\lxm\Anaconda3\Lib\site-packages\matplotlib\mpl-data目录下的文件matplotlibrc，用记事本或spyder打开，修改下面参数并保存。

```
font.family         : sans-serif
font.serif          : SimHei, DejaVu Serif, ……
font.sans-serif     : SimHei,DejaVu Sans, ……
axes.unicode_minus  : False  ## use unicode for the minus symbol
```

注意：去除前面的#号再保存

线条的类型

Linestyle 或ls	描述	Linestyle 或ls	描述
'- '	减号 画实线	':' ·	冒号 画短虚线
'--'	2个减号 画长虚线	'None' ' '	不画线
'-.'	减号加点号 画点划线		

常用的关键字：
color 颜色控制，color='green'
linestyle 线形控制 linestyle='dashed'
linewidth 线形控制 linewidth=4
标记控制：
marker='o' 标记符号
markfacecolor='blue' 标记颜色
markersize=12 标记大小

线条的标记

maker	描述	标记	描述
'o'	圆圈	'.'	点
'D'	菱形	's'	正方形
'h'	六边形1	'*'	星号
'H'	六边形2	'd'	小菱形
'_'	水平线	'v'	下三角形
'8'	八边形	'<'	左三角形
'p'	五边形	'>'	右三角形
','	像素	'^'	上三角形
'+'	加号	'\'	'
'None'	无	'x'	X

颜色控制

缩写	颜色
'r'	red
'g'	green
'b'	blue
'c'	cyan
'm'	magenta
'y'	yellow
'w'	white
'k'	black

numpy中快速生成数据的函数

- arange(起始点, 终止点, 步长)

  快速生成某个范围内的数据，默认间隔值为1

  如：np.arange(0,1,0.2)

  结果为array([0. , 0.2, 0.4, 0.6, 0.8])

- **linspace(起始值，终值，元素个数)**

  endpoint用于指定是否包括终值，默认为True

  如：np.linspace(10,100,6,endpoint=True)

  结果为array([10., 28., 46., 64., 82., 100.])

调整图例位置

- plt.legend(loc='String or Number', bbox_to_anchor=(num1, num2))

String	Number
upper right	1
upper left	2
lower left	3
lower right	4
right	5
center left	6
center right	7
lower center	8
upper center	9
center	10

bbox_to_anchor=(num1, num2))

- num1：控制水平相对位置
- num2：控制垂直相对位置

使用该参数，loc不起作用